Database Foreign Key Mapping

This document explicitly maps the **Foreign Key (FK) Relationships** tying together the 69 tables in the TalentOps Supabase database.

(Note: Foreign keys ensure referential integrity. They guarantee that an operation on one table won't leave "orphaned" data in another, blocking deletions if child rows still exist, or automatically cascading deletes across the tree.)

🏛 The Hierarchical Root Constraints

Your entire database is strictly structured around a multi-tenant hierarchy. Almost every data row stems down from the master `organizations` and `auth.users` tables.

1. The Tenant Root (`org_id`)

References: `orgs.id` (or `organizations.id` depending on the migration phase). This is the single most important foreign key in your database. It secures data so that one company cannot query another company's records.

Tables Restricted by this Foreign Key:

- `profiles` -> Every user is locked to an org.
- `departments`, `teams` -> Every group is locked to an org.
- `projects`, `tasks`, `task_steps` -> All work items are locked to an org.
- `attendance`, `leaves`, `payroll`, `payslips` -> All HR data is locked to an org.
- `conversations` -> Chat groups are locked to an org.
- `announcements` -> Company broadcasts are locked to an org.

2. The Identity Root (`user_id` / `employee_id`)

References: `profiles.id` (which in turn references the secure `auth.users.id` Supabase master table). This key connects all application actions back to a specific, authenticated human being.

Tables Restricted by this Foreign Key:

- `attendance`, `leaves`, `time_logs`, `timesheets` *(via `employee_id`)*
- `employee_finance`, `payroll`, `employee_skills` *(via `employee_id`)*
- `task_submissions` *(via `student_id` / user)*
- `messages`, `attachments`, `message_reactions` *(via `user_id` & `sender_user_id`)*
- `conversation_members` *(via `user_id`)*
- `project_members`, `team_members` *(via `user_id` / `employee_id`)*
- `poll_votes` *(via `user_id`)*
- `announcements`, `conversations` *(via `created_by`)*

🏗 Functional Module Constraints

Within the specific feature modules, tables constrain each other to maintain logical flow.

The Work Module (`project_id` & `task_id`)

References: `projects.id` and `tasks.id`. Ensures that you cannot submit work for a task that doesn't exist, nor add a task to a deleted project.

- `tasks` -> References `projects(id)`
- `project_members` -> References `projects(id)`
- `task_steps` -> References `tasks(id)`
- `task_submissions` -> References `tasks(id)`
- `task_skills` -> References `tasks(id)`
- `task_evidence`, `task_audit` -> References `tasks(id)`

The Communication Module (`conversation_id` & `message_id`)

References: `conversations.id` and `messages.id`. Ensures chat logs don't exist without their parent chat rooms.

- `conversation_members` -> References `conversations(id)`
- `conversation_indexes` -> References `conversations(id)`
- `messages` -> References `conversations(id)`
- `attachments` -> References `messages(id)`
- `message_reactions` -> References `messages(id)`
- `messages(reply_to_id)` -> Self-references `messages(id)` (for thread replies)

The Skills Engine Module (`skill_id`)

References: `skills_master.id`. A master dictionary pattern preventing users from typing random skill strings. Employees and tasks must select an explicitly defined skill ID from the master list.

- `employee_skills` -> References `skills_master(id)`
- `task_skills` -> References `skills_master(id)`
- `task_submission_skills` -> References `skills_master(id)`

🚫 Standard Unlinked Tables (Dictionaries & Standalones)

A small group of root tables have ZERO outbound foreign keys because they are the absolute "start" of a data chain, or standalone modules.

Master Dictionaries (Only Referenced BY others):

- `orgs`
- `skills_master`
- `policies` (Usually standalone documents)

(Note: Deleting a row from a master table like `skills_master` will currently throw a painful Postgres `Foreign Key Violation` error if any active `task_skills` are connected to it, preventing you from accidentally breaking existing logic).